



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: DISEÑO Y ESTRUCTURA DE COMPUTADORAS	Unidad #2 Unidades de procesamiento y juego de instrucciones.	Tema: Uso de Lenguaje ensamblador con C en Sistemas GNU/Linux.
	Semana 10	

Introducción:

Bienvenidos al material de lectura de la Unidad #2 de la materia Diseño y Estructura de Computadoras. En este material exploraremos el Uso de Lenguaje ensamblador con C en Sistemas GNU/Linux. Se presentará un repaso de los conceptos básicos del lenguaje de programación C (variables, estructuras de control y cíclicas, etc) y se comparará con el lenguaje ensamblador, para diferenciar la forma en que funciona cada uno de ellos, y posteriormente describir la manera que se pueden integrar ensamblador dentro de un programa escrito en lenguaje C.

Sección 1:

Antes de comenzar, asegúrate de tener papel y lápiz a mano para las actividades interactivas.

Conceptos de programación en ensamblador y C

Acceso a datos

Para acceder a datos de memoria en ensamblador, como sucede en los lenguajes de alto nivel, lo haremos por medio de variables que deberemos definir previamente para reservar el espacio necesario para almacenar la información. En la definición podemos especificar el nombre de la variable, el tamaño y un valor inicial; para acceder al dato lo haremos con los operandos de las instrucciones especificando un modo de direccionamiento determinado.

Por ejemplo, `var dd 12345678h` es la variable con el nombre `var` de tamaño 4 bytes inicializada con el valor `12345678h`. `mov eax, dword[var]` es la instrucción en la que accedemos a la variable `var` utilizando direccionamiento directo a memoria, en la que especificamos que queremos acceder a 4 bytes (`dword`) y transferimos el contenido, el valor `12345678h`, al registro `eax`.

En ensamblador hay que estar muy alerta cuando accedemos a las variables que hemos definido. Las variables se guardan en memoria consecutivamente a medida que las declaramos y no existe nada que delimite las unas de las otras.

Ejemplo

```
.data
var1 db 0           ;variable definida de 1 byte
var2 db 61h         ;variable definida de 1 byte
var3 dw 0200h       ;variable definida de 2 bytes
var4 dd 0001E26Ch   ;variable definida de 4 bytes
```

Las variables se encontrarán en memoria tal como muestra la tabla 7.

Tabla 1

Variables en memoria

Dirección	Valor
var1	00h
var2	61h
var3	00h
	02h
var4	6Ch
	E2h
	01h
	00h

Nota: La tabla describe las variables y sus valores en memoria. Fuente: Orenge & Manonellas (2011).

Estructuras de datos

Veamos cómo acceder a vectores y matrices utilizando lenguaje C y lenguaje de ensamblador. Definimos en C un vector y una matriz y hacemos la suma de los elementos.

```
int main(){

    int vec[6]={1,2,3,4,5,6},mat[2][3]={{1,2,3},{4,5,6}};
    int i,j, sumaVec=0, sumaMat=0;

    for (i=0;i<6;i++) sumaVec=sumaVec+vec[i];

    for (i=0;i<2;i++){
        for(j=0;j<3;j++) sumaMat=sumaMat+mat[i][j];
    }
}
```

A continuación, veremos cómo hacer lo mismo con lenguaje de ensamblador, utilizando diferentes variantes del direccionamiento indexado (el formato general es [dirección + Registro Base + Registro Índice * escala]):

```

vec dd 1,2,3,4,5,6
mat dd 1,2,3
    dd 4,5,6

;recorremos el vector para sumarlo
mov esi,0 ;esi será el índice para acceder a los datos
mov eax,0 ;eax será donde guardaremos la suma
loop_vec:
    add eax, dword[vec+esi*4] ;multiplican el índice por 4
    inc esi ;porque cada elemento ocupa 4 bytes.
    cmp esi, 6 ;comparamos con 6 porque es el índice del primer
    ;elemento fuera del vector.
    jl loop_vec

;recorremos la matriz con un solo índice para sumarla
mov esi,0 ;esi será el índice para acceder a los datos
mov ebx,0 ;ebx será donde guardaremos la suma
loop_mat:
    add ebx, dword[mat+esi*4] ;multiplican el índice por 4
    inc esi ;porque cada elemento ocupa 4 bytes.
    cmp esi, 6 ;comparamos con 6 porque es el índice del primer elemento
    ;fuera de la matriz, la matriz tiene 2*3=6 elementos.
    jl loop_mat

;recorremos la matriz con dos índices para sumarla
mov esi,0 ;esi será el índice de la columna
mov edi,0 ;edi será el índice de la fila
mov ebx,0 ;ebx será donde guardaremos la suma

loop_mat2:
    add ebx, dword[mat+edi+esi*4] ;multiplican el índice de la columna
    inc esi ;por 4 porque cada elemento ocupa 4 bytes.
    cmp esi, 3 ;comparamos con 3 porque son los elementos de una fila.
    jl loop_mat2
    mov esi, 0
    add edi, 12 ;cada fila ocupa 12 bytes, 3 elementos de 4 bytes.
    cmp edi, 24 ;comparamos con 24 porque es el primer índice
    ;fuera de la matriz.
    jl loop_mat2

```

Operaciones aritméticas

En este subapartado hemos de tener presente que en C podemos construir expresiones utilizando variables, constantes y operadores en una misma sentencia; en cambio, hacer lo mismo en ensamblador implica escribir una secuencia de instrucciones en la que habrá que hacer las operaciones una a una según la prioridad de los operadores dentro de la sentencia.

Ejemplo

Sentencia que podemos expresar en lenguaje C:

```
r=(a+b)*4 / (c>>2);
```

Traducción de la sentencia en lenguaje de ensamblador:

```
mov eax, [a]
mov ebx, [b]
add eax, ebx ; (a+b)
imul eax, 4 ; (a+b)*4
mov ecx, [c]
sar ecx, 2 ; (c>>2)
idiv ecx ; (a+b)*4 / (c>>2)
mov [r], eax ; r=(a+b)*4 / (c>>2)
```

Control de flujo

En este subapartado se presentan cómo las diferentes estructuras de control del lenguaje C se pueden traducir a lenguaje de ensamblador.

Estructura if

Estructura condicional expresada en lenguaje C:

```
if (condición) {
    bloque de sentencias
}
```

Ejemplo

```
if (a > b) {
    maxA = 1;
    maxB = 0;
}
```

Se puede traducir a lenguaje de ensamblador de la manera siguiente:

```
mov rax, qword [a]      ;Se cargan las variables en registros
mov rbx, qword [b]

cmp rax, rbx            ;Se hace la comparación
jg cierto               ;Si se cumple la condición, salta a la etiqueta cierto
jmp fin                ;Si no se cumple la condición, salta a la etiqueta fin

cierto:
    mov byte [maxA], 1 ;Estas instrucciones solo se ejecutan
    mov byte [maxB], 0 ;cuando se cumple la condición

fin:
```

Estructura while

Estructura iterativa controlada por una condición expresada al principio:

```
while (condiciones) {
    bloque de sentencias que ejecutar
}
```

Ejemplo

```
resultado=1;
while (num > 1){ //mientras num sea mayor que 1 hacer ...
    resultado = resultado * num;
    num--;
}
```

Se puede traducir a lenguaje de ensamblador de esta manera:

```
mov rax, 1              ;rax será [resultado]
mov rbx, qword [num]    ;Se carga la variable en un registro
while:
    cmp rbx, 1          ;Se hace la comparación
    jg cierto           ;Si se cumple la condición (num > 1) salta a cierto
    jmp fin            ;Si no, salta a fin
cierto:
    imul rax, rbx        ;rax = rax * rbx
    dec rbx
    jmp while
fin:
    mov qword [resultado], rax
    mov qword [num], rbx
```

Estructura for

Estructura iterativa, que utiliza la orden for:

```
for (valores iniciales; condiciones; actualización) {  
    bloque de sentencias que ejecutar  
}
```

Ejemplo

```
resultado=1;  
for (i = num; i > 1; i--)  
{  
    resultado=resultado*i;  
}
```

Se puede traducir a lenguaje de ensamblador de esta manera:

```
mov rax, 1          ;rax será [resultado]  
mov rcx, qword [num] ;rcx será [i] que inicializamos con [num]  
  
for:  
    cmp rcx, 1      ;Se hace la comparación  
    jg cierto       ;Si se cumple la condición, salta a cierto  
    jmp fin  
cierto:  
    imul rax,rcx  
    dec rcx  
    jmp for  
fin:  
    mov qword [resultado], rax  
    mov qword [i], rcx
```

Al salir de la estructura iterativa, no actualizamos la variable num porque se utiliza para inicializar i pero no se cambia.



Actividad interactiva

Como se puede traducir a lenguaje ensamblador el siguiente código C (estructura if-else).

Ejemplo

```
if (a > b) {  
    max = 'a';  
}  
else { // (a <= b)  
    max = 'b';  
}
```



Actividad interactiva

Como se puede traducir a lenguaje ensamblador el siguiente código C (estructura do-while).

Ejemplo

```
resultado = 1;  
do {  
    resultado = resultado * num;  
    num--;  
} while (num > 1)
```